

2026 창의설계경진대회 · 광진구 V2N 교통 시뮬레이션

V2N 중앙 제어 알고리즘

사고 대응 · 실시간 경로 재계산 · 정체 예측 우회 · 신호 시간 배분

18,521

차도 구간(edge)

197

신호 교차로

7,200

시간당 차량

~900

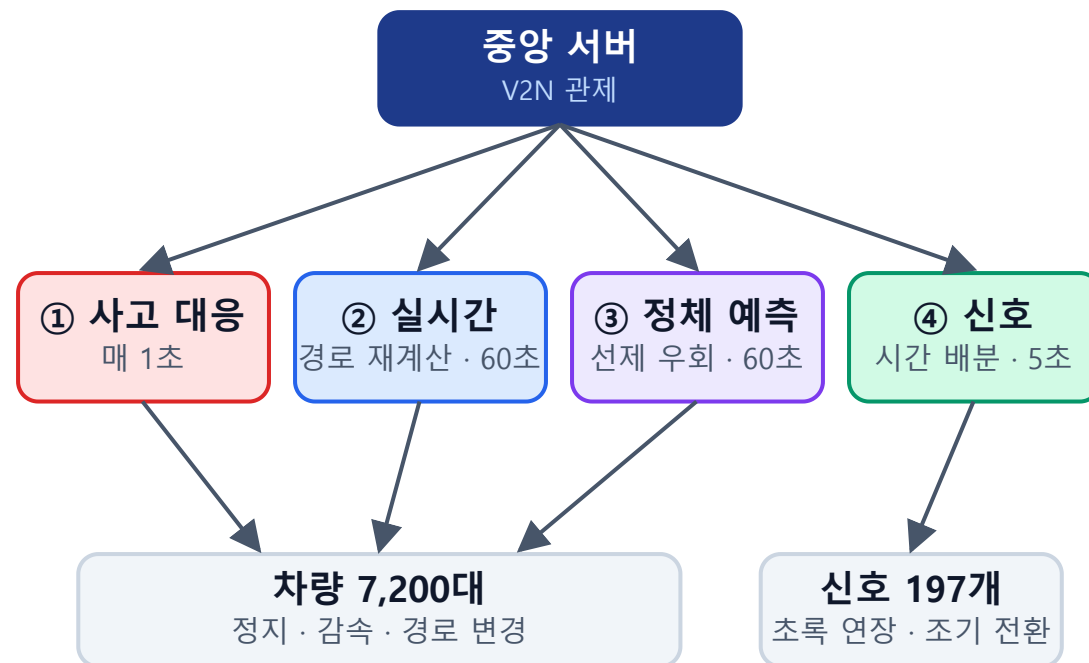
시간당 보행자

01 전체 구조 — 관제탑 하나가 도시 전체를 본다

V2N(Vehicle-to-Network): 모든 차가 이동통신망으로 중앙 서버에 위치·목적지를 보내고, 서버가 도시 전체를 보고 명령한다.

입력	차량 위치·경로(V2N), CCTV 9곳 감지, 도로망 전역 사고 신고
판단	4개 알고리즘 — 사고 대응 / 실시간 재계산 / 정체 예측 / 신호 배분
출력	차별 명령(정지·감속·경로 변경), 신호 초록 시간 변경
주기	매 1초 판단, 무거운 계산은 5초·60초 주기로 나눔

구현: SUMO(오픈소스 교통 시뮬레이터) + TraCI(실행 중 시뮬레이션을 파이썬으로 제어하는 API)



보조: 180초 이상 낀 차 견인(현실 보정, 비교 양쪽 동일)

매 1초 처리 순서



판단과 실행 분리

판단 규칙(`decision_policy.py`)은 SUMO를 호출하지 않는 순수 함수. 실제 CCTV 입력으로 바뀌 끼워도 그대로 동작하는 구조.

한 차에 명령 하나

여러 위험이 겹쳐도 우선순위 **정지 > 우회 > 감속**으로 하나만 배정. 정지할 차는 우회를 계산할 여유가 없다고 본다.

계산 비용 분산

전역 경로 탐색은 차당 약 10ms. 7,200대를 매 초 돌리면 멈추므로 60초 주기, 선제 우회는 사이클당 최대 120대.

03 감지 — 무엇을 보고 판단하나

감지	범위	조건
사고	도로망 전역	사고 도로 발생 즉시(CCTV 9곳 + 그 밖은 V2N 차량 신고)
보행자	CCTV 9곳 + 어린이보호구역	구역 도로 위 보행자
정체	CCTV 9곳	구역 도로 평균 속도 < 2 m/s
실물 보행자	신양초 사거리	RC카 보드의 LED 보행자(YOLO 인식)

CCTV 9곳은 데이터로 선정

1. 연결 도로 수(연결도) 상위 30개 교차로
2. 시뮬레이션 1시간 실측 통행량 상위 9곳 확정

실험용 사고 규칙 (스트레스 테스트)

- 30초마다 1건, 30초 지속 — 새 사고가 나면 이전 사고는 풀림
- 위치: 지금 도로 위 모든 차의 남은 경로에 **가장 많이 들어 있는 도로** = 앞으로 가장 많은 차가 지나갈 도로
- 제외: 20m 미만 도로, 어린이보호구역, 최근 사고 5건 도로
- 사고 도로 위 차는 멈춘다 — **중앙 제어가 있든 없든 동일**

실제 사고 빈도가 아니라 사고 대응 알고리즘이 가장 많이 작동하는 최악 조건. 감지는 완벽하다고 가정(영상 오인식은 모델링하지 않음).

04 알고리즘 ① 사고 대응 — 위치에 따라 세 가지 명령



칸(hop) = 교차로와 교차로 사이 도로 한 토막

정지 — 사고 도로 위

진로가 물리적으로 막힘. 목표 속도 0.
사고가 풀리면 다음 스텝부터 자동 해제(명령 유효 2초, 매 초 재판단).

감속 — 1~2칸 앞

빠질 갈림길이 거의 없음. 18 km/h로
늦춰 사고 지점에 안전하게 접근 — 급정거·추돌 예방.

우회 판단 — 3~5칸 앞

남은 경로에 사고 도로가 있는 차만 대상. 사고 도로를 피하는 경로를 계산하
되, 다음 장의 대기 비교를 통과해야 실
제로 우회.

기준값: NEARBY_HOPS=2, REROUTE_HOPS=5, 감속 5.0 m/s — core/cctv_detector.py, core/decision_policy.py

05 알고리즘 ① 사고 대응 — 우회할까, 기다릴까

우회가 항상 빠르지 않다. 30초 뒤 풀릴 사고 때문에 간선도로에서 골목으로 빠지면 오히려 늦는다(실측 예: 137초 → 227초).

기다리면 $ETA_{대기} = \max(\text{사고 도로 도착 시각}, \text{사고 해제 시각}) + \text{사고 도로 통과(제한속도)} + \text{나머지 경로}$

돌아가면 $ETA_{우회} = \text{사고 도로에 큰 통행시간 벌점을 준 뒤 SUMO 라우터가 찾은 최단시간 경로}$

$ETA_{우회} < ETA_{대기}$ 일 때만 우회 · 아니면 원래 경로 유지(사고 끝까지 재계산 안 함)

- 도로 통행시간은 SUMO가 매 순간 측정한 현재 평균 속도 기준
- 멈춘 도로는 추정치가 수만 초로 튀므로 도로당 최대 300초로 자름
- 벌점 창은 30초 — 실측 결과 2~10초는 라우터가 무시함(이 버그를 찾아 고침)

관찰 결과 (대시보드 10배속)

18/18 우회한 차 전부 대기보다 빨랐다
평균 13초, 최대 25초 단축

20대 원래 경로 유지
우회했으면 중앙값 33초 손해

도시 전체 지표(5 seed, 컴 vs 끄)로는 seed마다 방향이 갈려 효과 판별 불가 — 사고 한 건이 영향을 주는 차는 수십 대, 도시 전체는 7,200대.

`core/vehicle_controller.py` `_reroute_around()`

알고리즘 ② 실시간 경로 재계산

사고 대응은 사고 주변만 본다. 도시 곳곳에서 서서히 쌓이는 정체는 이 알고리즘이 맡는다.

- 1 모든 차의 마지막 계산 시각을 기록
- 2 **60초가 지난 차**만 경로 재계산 — 차마다 출발 시점이 달라 계산이 시간에 고르게 퍼짐
- 3 기준: 각 도로의 **지금** 평균 통행시간(SUMO 실측) → 최단 시간 경로
- 4 실물 RC카(보드 위)는 제외 — 경로는 RC카 제어기가 정함

일반 내비와 다른 점

	개인 내비	V2N 중앙 제어
목표	내 차 최단	도시 전체 흐름
정보	과거·표본 속도	모든 차의 현재 위치·경로
물림	모두 같은 길로	③에서 분산
신호	관여 못 함	④에서 함께 조정

60초 주기는 계산 비용(차당 약 10ms × 7,200대)과 정체 변화 속도를 고려한 설계값. `core/central_server.py`

07 알고리즘 ③ 정체 예측 선제 우회

- 1 **예측:** 60초마다 모든 차의 남은 경로를 따라가며 **앞으로 90초 안에** 각 도로에 도착할 차 수를 센다
- 2 **용량 비교:** 도로 용량 = 차선 수 × 0.4대/초(시간당 1,440대/차선) × 90초. 넘으면 "넘칠 도로"
- 3 **대상 선정:** 그 도로까지 **45초 이상 여유**가 있는 차만 — 이미 가까이 온 차는 대안도 같이 막혀 있다
- 4 **급한 순서**로 사이클당 최대 120대 경로 재계산(넘칠 도로에 60초 벌점)
- 5 **몰림 방지:** 같은 대체 도로에 차선당 15대가 넘게 배정되면, 다음 차는 그 도로도 피해 다시 계산(그 차에만 적용되는 개별 벌점)

몰림(herd effect) 실측과 개선

개선 전  97%

개선 후  56%

한 사이클 우회 차 중 가장 많이 몰린 대체 도로로 간 비율(73대 중 71대 → 56%)

트레이드오프 (seed 1~3, 컴 vs 끄)

총 통행시간 **+0.9~2.7%** · 누적 대기 **-0.7~8.8%**
미리 돌아가서 거리는 늘고 막혀 서 있는 시간은 준다.

머신러닝이 아니라 계산 — 중앙이 모든 차의 경로를 알기 때문에 가능. `core/predictive_reroute.py`

08 알고리즘 ④ 신호 시간 배분

5초마다 신호 197개 전부: 지금 초록 방향과 다음 방향의 대기 차량 수(속도 0.1 m/s 미만)를 비교한다.

조건	행동
초록이 최소 시간 전	절대 안 건드림 — 보행자 안전 시간 보장
다음 방향 대기 > 지금 방향 × 1.8	조기 전환 — 2초 뒤(급정거 방지)
지금 방향 대기 ≥ 3대	연장 = 4초 + 대기 1대당 1초 (최대 20초, 최대 시간 이내)
그 밖	원래 신호 그대로

max-pressure(대기열 압력 비교) 신호 제어의 단순화 버전. `core/signal_controller.py`

해 보고 버린 것 — 전부 실측으로 판단

- 빈 방향은 최소 시간도 건너뛰기 → 효과 없음
- 하류 도로가 꽉 차면 연장 안 함 → 그리드락 +45%
- 짧은 진입로는 상류 대기열까지 세기 → 전반 악화

교훈: 신호 로직을 도시 전역으로 바꾼 시도는 세 번 모두 실패했고, 도로망 구조(쪼개진 사거리 합치기)를 고친 것이 가장 효과가 컸다(견인 -80%, 만성 정체 -75%).

보조 — 견인(현실 보정)

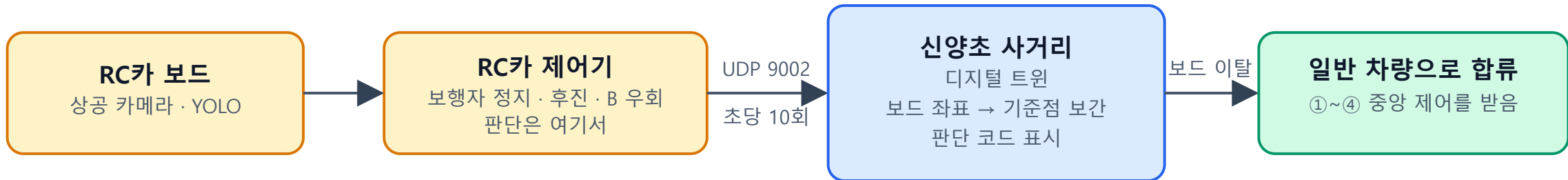
20초마다 점검해 **연속 180초 이상 멈춘 차**를 도로에서 치운다. 실제 도로라면 견인차가 오는 상황.

- 180초 = 신호 한 주기(90초)의 2배 → 신호 대기과 확실히 구분
- 없으면: 충돌로 낀 차가 SUMO 기본 순간이동(300초) 뒤 같은 자리에 다시 갇히는 무한 반복(실측)
- **중앙 제어 있음/없음 양쪽에 똑같이 적용** — 알고리즘 효과로 착각되지 않게
- 실물 RC카는 견인하지 않음(실물이 서 있으면 화면도 서 있어야 함)

정체를 재는 지표

견인	180초 이상 낀 차 수 — "막힘"의 가장 직접적인 지표
그리드락	한 교차로로 들어오는 2개 이상 방향이 동시에 60초 넘게 정지
만성 정체	한 도로 평균 2 m/s 이하가 120초 넘게 지속
TSTT	모든 차가 도로 위에 있던 시간 합(도착 못 한 차 포함)

10 실물 연동 — 현장 자율 판단과 도시 중앙 제어의 연결



연동은 차량 → SUMO 한 방향(팀 결정): 실차 검증된 판단을 시연 직전에 바꾸지 않는다

좌표 변환

보드(1.34 × 0.91 m)와 실제 사거리는 모양이 달라 비례 변환 불가. 출발·사거리 입구·보행자 선·도착 기준점끼리 짝지어 구간별 보간 → 실물이 LED 앞에서 서면 SUMO 차도 보행자 앞에서 선다.

보행자 = 사고와 같은 흐름

보드 LED 보행자가 인식되면 신양초 출구 도로에 위험을 켜고, 주변 SUMO 차는 ① 사고 대응 규칙대로 정지·우회한다.

도시 교통으로 합류

보드 주행이 끝난 RC카는 같은 자리·같은 모습의 일반 차량이 되어 1.5km 이상 떨어진 목적지까지 중앙 제어를 받으며 달린다.

11 설계 원칙과 검증 방식

원칙

- **표준 기법 사용:** 우회는 직접 길찾기 대신 SUMO 라우터에 벌점을 주는 표준 incident-avoidance 방식 → 도로망 어디서나 동작
- **판단과 실행 분리:** 규칙은 SUMO 없이 테스트(단위 테스트 17개)
- **보수적 명령:** 가장 강한 명령은 정지, 목적지는 절대 바꾸지 않음
- **안전 시간 불가침:** 신호 최소 시간은 건드리지 않음
- **재현 가능:** 도로망은 설정 파일 하나로 빌드, 차량·보행자·사고 난수 모두 seed로 고정

검증 방식

- 같은 seed로 **중앙 제어 없음 vs 컴 비교**, seed 5쌍 + 통계 검정 (대응 t-검정·Wilcoxon)
- 사고는 양쪽 조건 모두 같은 규칙으로 발생, 차이는 알고리즘뿐
- 평균뿐 아니라 "몇 개 seed에서 나아졌나"를 함께 봄
- 스스로 찾아 고친 버그: 우회 벌점 창이 너무 짧아 무시됨, 보행자 크기 왜곡, 사고 난수 미고정, 신호 방향 오인식, 우회 몰림 97%

비교 결과는 별도 자료 "통계 비교표" 참고.

① 사고 대응

사고 도로 정지 · 1~2칸 감속 · 3~5칸 우회 판단
우회가 대기보다 빠를 때만 우회

② 실시간 재계산

60초마다 모든 차를
현재 통행시간 기준 최단 경로로

③ 정체 예측

90초 뒤 넘칠 도로 예측 →
여유 있는 차만 분산 우회

④ 신호 배분

대기열 비교로 연장·조기 전환
최소 시간은 보장

관제탑 하나가 도시 전체를 보고, 현장의 실물 차량 판단까지 하나의 화면으로 잇는다.